Group Assignment — AI System Design and Minimum Viable Workflow

Design, build, and evaluate an AI-powered solution to a problem of your group's choosing

AI Agents for Managers · Group assignment
Present in class: final session (2-minute group presentation + 1 minute of questions) · **Individual reflection due:** after the presentation, deadline announced in class · **Submit via:** the course site

1 · Introduction

This is the **capstone** assignment of the course, and you will complete it **as a group**. You will apply everything you have learned — from designing AI workflows and building chatbots to working with agentic coding tools and evaluating AI outputs — to a problem of your group's own choosing.

The goal is simple: find a real problem, design an AI-powered solution, build a working prototype, and evaluate whether it actually works. The problem can come from a group member's personal life or professional experience, a product idea, a startup concept, or anything else meaningful to your group. The only requirement is that it should be a task where an AI agent or AI-powered workflow could provide genuine value.

Your group will present its work in a **two-minute pitch in class**, in which **every member of the group speaks**, followed by **one minute of questions** directed at different members of the group. Afterwards, **each group member** submits a **one-page individual reflection** on their own experience. The project and the presentation are graded **as a group**: everyone in the group receives the same grade for them, based on the whole group's performance in the presentation and on all of the questions. The reflection is graded **individually**.

Learning objectives

By completing this assignment, you will:

- Identify a real business or personal problem where an AI workflow could add value
- Design a system diagram that maps out the components and data flow of your AI solution
- Build a working prototype using an agentic coding tool (such as Claude Code, Claude Cowork, OpenAI Codex, Antigravity or similar) **or** AgentLab
- Evaluate your prototype's outputs against what "good" looks like for your use case
- Reflect critically on what works, what doesn't, and how the system could be improved

Prerequisites

Before starting this assignment, make sure your group has:

1. **An agentic coding tool.** You need access to at least one tool that can write and execute code based on your instructions. Options include:
   - **Claude Code or Claude Cowork** (recommended) — install via terminal (`curl -fsSL https://claude.ai/install.sh | sh`) or download the Claude Desktop App, which includes both. Make sure to log in with your paid Claude account.
   - If you cannot install locally, Claude Code is also available in the browser: see *https://tessl.io/blog/anthropic-brings-claude-code-to-the-web-and-mobile/*
   - Other agentic tools (OpenAI Codex, Gemini CLI Agent, Cursor, Windsurf, Replit Agent, etc.) are also fine.
2. **AgentLab (optional alternative).** If you prefer to build your workflow using the visual, no-code AgentLab tool from Assignments #1 and #2, that is acceptable — but note that you will likely be more limited in what you can build.
3. **A way to draw system diagrams.** You can use any tool you like: a whiteboard and photo, pen and paper, PowerPoint/Google Slides, draw.io, Miro, Excalidraw, or even ask an LLM to generate one for you (but it should be the one you actually built!).

> **Safe setup for agentic coding tools.** When using Claude Code or similar agentic tools that execute code on your machine, it's good practice to work in a dedicated, empty folder. For additional safety guidance, see *https://www.anthropic.com/engineering/claude-code-sandboxing*. When starting Claude Code in a terminal, first navigate to an empty project folder before running `claude` — this keeps the agent focused on your project files.

2 · The assignment

This assignment has three phases: **Design, Build, and Evaluate.** You will document all three phases in your submission.

Phase 1 · Choose a problem and design your system

Step 1.1 — Identify your problem. Pick a problem that matters to you. It should be something where an AI-powered workflow could provide genuine value — not just a toy example. Good problems are ones where you can clearly articulate what "success" looks like.

Examples of good problems:

- "I get dozens of customer support emails a day and want to auto-draft responses based on our FAQ and past answers."
- "I want to build a tool that takes meeting transcripts and automatically produces action items, assigns owners, and drafts follow-up emails."
- "I want to create an AI-powered financial news summarizer that filters and prioritizes articles relevant to my portfolio."

- "I run a small e-commerce shop and want to auto-generate product descriptions from supplier spec sheets."
- "I want a personal AI assistant that triages my unread emails every morning and drafts replies for the routine ones."
- "I want to build an interview prep tool that generates practice questions based on a job description and grades my answers."

Step 1.2 — Draw a system diagram. Before you build anything, draw a diagram of how your AI system should work. It should show:

- **Inputs:** What data or information goes into the system? (e.g., emails, documents, user prompts, database records)
- **Processing steps:** What does the AI do with the inputs? Are there multiple steps? Is there routing or branching logic? Are there multiple LLM calls?
- **Tools and data sources:** Does the system use RAG, web search, APIs, file access, or other tools?
- **Outputs:** What does the system produce? (e.g., a document, a classification, a recommendation, an email draft)
- **Validation or human-in-the-loop steps:** Is there a quality check, an LLM-as-a-Judge, or a point where a human reviews the output?

Think about the agentic design patterns discussed in class: sequential chains, parallelization, orchestrator-worker, and hierarchical designs. Which pattern best fits your problem? The important thing is that **someone else could understand and build this workflow based on your design.**

> 💡 **Tip.** Your system diagram doesn't have to be pretty — it has to be clear. A hand-drawn sketch on paper that clearly shows the flow of data is better than a polished diagram that's confusing. You can also ask Claude or ChatGPT to generate one for you, or a description you can then draw.

Phase 2 · Build your prototype

Step 2.1 — Choose your building approach. You have two main options:

Option A — Agentic coding tool (recommended). Use Claude Code, Claude Cowork, OpenAI Codex, or a similar agentic coding tool to build your workflow. The key idea is that you *describe* what you want the tool to build, and it writes the code for you. You do not need to know how to code — the agentic tool does the programming. Your job is to:

- Clearly describe what you want built
- Test the outputs and iterate when something is wrong
- Ask the tool to explain what it built and how the components connect
- Guide the tool through revisions until the prototype works — this may involve copying output into other tools as instructed by the coding agent, or providing additional information; figuring out how to use these tools, even when you are uncertain, is a key skill

Option B — AgentLab. If you prefer the visual, no-code approach from Assignments #1 and #2, you may use AgentLab. You will have more direct control over the structure, but the range of what you can build will be more limited.

> ⚠ **You are the manager, not the programmer.** Regardless of which tool you use, the point of this assignment is *not* to write code yourself. The point is to act as a manager and product designer: you define the problem, specify the requirements, direct the AI tool to build a solution, evaluate whether the solution works, and iterate to improve it. This mirrors the real-world skill of managing AI-powered processes.

Step 2.2 — Build and iterate. Work with your chosen tool to build the prototype. As you build, keep track of:

- What instructions you gave the tool (your key prompts)
- What worked on the first try and what required iteration
- Any errors, misunderstandings, or unexpected behaviors
- How you guided the tool to fix problems

Step 2.3 — Understand what was built. Once your prototype is working, understand its architecture — ask the agentic tool to explain it, or, in AgentLab, use the **Inspector** and the **exported report** to see the same thing. You should be able to answer:

- What are the main components or modules?
- How does data flow between them?
- Where are LLM calls being made, and what prompts are being used?
- Does the final architecture match your original system diagram, or did it evolve?

> 📝 **Note.** If you used an agentic coding tool, have it generate an updated system diagram or architectural summary for you (AgentLab's exported report includes a diagram of your flow). Compare this to your original design. Differences between your intended design and the actual implementation are interesting and worth discussing in your reflection.

Phase 3 · Evaluate your prototype

Building a prototype is not enough — you also need to evaluate whether it actually works well. This is where the evaluation skills from class (and the in-class LLM evals exercise) come in.

Step 3.1 — Define what "good" looks like. Before you test, write down what a good output looks like for your specific use case. For example:

- If your tool drafts emails: What makes a draft good? (correct tone, includes all key points, appropriate length, no hallucinated details)
- If your tool classifies inputs: What is the correct classification for your test cases?

- If your tool generates recommendations: What criteria distinguish a good recommendation from a bad one?

Hand-labeling specific examples is often a good way to formalize your intuition for subjective assessments and also provides you with a data set against which to test changes in your workflow.

Step 3.2 — Test with multiple inputs. Run your prototype with at least **5 different inputs** (more is better). These should include:

- **Easy cases** where you expect the system to perform well
- **Edge cases** that might be tricky (ambiguous inputs, unusual formats, missing information)
- **Adversarial cases** that try to break the system (unreasonable requests, contradictory inputs, inputs outside the expected domain)

For each test, document the input, the output, and your assessment of whether the output was good, acceptable, or poor.

Step 3.3 — Evaluate and iterate. Based on your testing:

1. Identify the most common failure modes. Where does your system break down?
2. Try to improve it. Can you adjust prompts, add validation steps, or modify the workflow to fix the failures?
3. Re-test after making changes. Did your fixes improve things? Did they cause regressions elsewhere?
4. Document one full cycle of this **measure** → **diagnose** → **improve** → **re-measure** loop.

> 🧪 **Tip.** You can use an LLM-as-a-Judge to help evaluate outputs at scale, just as we discussed in class. For example, you could ask Claude to rate outputs on specific criteria and compare its assessments to your own. This is not required, but if you do it, document what you found — it will strengthen your submission.

3 · Deliverables

There are **two** things to hand in: **two slides per group**, and **one reflection per person**.

3.1 · Two presentation slides — present in class

Your group will give a **two-minute pitch** in class, and **every member of the group must present part of it**. Prepare **exactly two slides**:

1. **Slide 1 — Your system diagram.** Show the design of your AI workflow: what goes in, what processing happens, and what comes out.
2. **Slide 2 — A screenshot or demo of your working prototype.** Show that it actually runs and produces output. A screenshot of a working interaction is sufficient; you do not need to do a live demo.

Your verbal pitch should explain what problem you solved, why it matters, and what you learned — think of it as pitching your product to a room full of potential users or investors. Decide in advance who covers which

part, so that the hand-offs between speakers are smooth.

> ⚠ **Two minutes is short and the time limit will be tightly enforced.** The presentation stops at two minutes, whether or not you have finished. Practice as a group so you can cover the problem, the system, and what you learned in the time. Part of the skill is refining your pitch to make it concise.
>
> **Questions (1 minute).** After the pitch, there will be one minute of questions. Questions will be directed at **different members of the group**, not only the person who presented a given part, so **every member must be able to explain the details of the whole system**: the problem, the design, how it was built, and how it was evaluated. The group is graded on its performance across **all** of the questions.

3.2 · Individual reflection — due after the presentation

After the presentation, **each group member submits their own one-page reflection** on their experience. This is **individual work**: write it yourself, about what *you* did and learned. Your group's two slides already carry the problem statement and the system design, so the reflection should focus on **your role, how the system was built and tested, and what you learned.** Cover:

Cover	What to include
Your role	What you personally contributed to the design, the build, and the testing.
Evaluation	What "good" output meant for your system, what the testing showed, and one **measure** → **diagnose** → **improve** → **re-measure** cycle you were involved in.
Reflection	What you learned. Current limitations of the prototype. How you would improve it with more time. What was easy or hard about managing an agentic tool, and about building it as a group.

*Keep it to **one page**. The quality of your reflection matters far more than its length, and **longer reflections will be penalized**: distilling what matters is part of the skill.*

4 · Grading

This assignment is worth **30% of your course grade.** The first three criteria below are graded **as a group: every member of the group receives the same grade** for them, based on the group's presentation and its answers to all of the questions. The reflection is graded **individually**, based on your own one-page reflection. You will be evaluated on:

Criterion	Weight	What earns full marks
Problem & system design	25%	Clear, well-motivated problem. Diagram is logical and complete.
Working prototype	25%	Prototype runs and produces meaningful output. Evidence of testing and iteration.
Presentation & questions	25%	Clear, concise 2-minute pitch in which every member presents. Covers the problem, the system, and the takeaway across your two slides. Every member can answer questions about the whole system.
Reflection & insights (individual)	25%	Specific, original reflections on your own role and experience. Discusses limitations and improvements.

> 🎓 **Grading philosophy.** Full grades on the reflection require evidence of substantive effort: specific, original reflections tied to your own role, design and process — not generic observations that could apply to any project. A thoughtful analysis of a prototype that doesn't fully work is worth more than a superficial reflection on one that does. The goal is to demonstrate that you engaged deeply with the process of designing, building, and evaluating AI systems.

5 · Timeline and submission

Date	Milestone
As early as possible	Choose your problem and have your system diagram ready. Start building.
Final session — in class	Deliver your 2-minute group presentation using your two slides, followed by 1 minute of questions.
After the presentation (deadline announced in class)	Submit your group's two slides and your own one-page individual reflection via the course site.

6 · Tips for success

Choosing your problem

- **Start with a real pain point.** The best projects come from genuine frustration with a manual process or a gap in existing tools. If your group cares about the problem, your presentation and reflections will be more engaging.

- **Scope it down.** Your time is limited. Pick a specific, well-defined slice of a larger problem rather than trying to solve everything. A tool that does one thing well is better than a tool that does five things poorly.
- **Think about evaluation upfront.** Choose a problem where you can clearly tell whether the AI output is good or bad. If you can't evaluate it, you can't improve it.

Working with agentic coding tools

- **Be specific in your instructions.** Instead of "build me an email tool," say "build a Python script that reads emails from a CSV file, classifies them as urgent or non-urgent, and drafts a short reply for each."
- **Iterate in small steps.** Get the basic version working first, then add complexity. Don't try to build the entire system in one prompt. With AI tools, "re-work" is easy.
- **When something breaks, describe the problem clearly.** Tell the tool what happened, what you expected, and ask it to fix the specific issue. Copy-pasting error messages is very helpful.
- **Ask the tool to explain itself.** After building, ask "what is the architecture of what you just built?" or "can you draw a diagram of the modules?" This will help you understand and document the system.

Evaluation

- **Test the boundaries.** Don't just test with "happy path" inputs. Try unusual, incomplete, or adversarial inputs to find where the system fails.
- **Be honest about failures.** A thoughtful discussion of what went wrong is more valuable than pretending everything worked. The best evaluations are honest.
- **Show your improvement cycle.** Even one round of "I found this failure, I changed this prompt, and here's what happened" demonstrates the eval-driven development process from class.

7 · Encouragement

This assignment is designed to be achievable regardless of your technical background. The entire point of agentic coding tools is that they enable people to build functional software by describing what they want in plain language. You do not need to understand the code that gets generated — you need to understand the problem, the design, and whether the output is good.

If you get stuck at any point, try the following, in order:

- Ask an LLM for guidance
- Ask your group members and classmates for help
- Request office hours

Getting help is encouraged — just make sure your individual reflection reflects your own thinking and your own experience.

The only good way to get an intuition for how AI agents work is to gain hands-on experience building with them. This assignment is your chance to go from someone who *uses* AI to someone who *builds and manages*

AI tools — which is exactly the transition this course is designed to enable.